HW4 Reinforcement Learning

Adrian Ramirez

October 2026

Question 1

(a)

Why does the value jump up for the last two rows in the rear (player sum 20 and 21)?

The value jumps up because the policy is being evaluated sticks only on 20 or 21. In addition, those are the states with very strong hands and where the player stops drawing. Therefore the dealer must reach 20 or blackjack to tie or beat them, so the player usually wins and the value is close to +1.

Why does it drop off for the whole last row on the left (dealer showing an ace)?

It drops off for the whole last row on the left because an ace is the dealer's best possible card. It can count as both 1 and 11, which leads to the dealer having a strong total on average. Whatever the player's cards are, the expected outcome is worse on average.

Why are the frontmost values (low player sums) higher in the upper diagrams (usable ace) than in the lower ones?

With a usable ace, the player is in an advantageous position and can't bust on the next card. If a hit would push the player over 21, the ace switches from 11 to 1 and the player keeps playing. This makes hitting from a lower sum much safer, giving the player more of a chance to reach 20 or 21.

(b)

Incremental Monte Carlo ES

Initialization: for every state $s \in \mathcal{S}$ and action $a \in \mathcal{A}(s)$:

- $Q(s,a) \leftarrow$ arbitrary
- $N(s,a) \leftarrow 0$ (this replaces Returns$(s,a) \leftarrow$ empty list)

Inside the loop, after computing G for the first visit of (S_t, A_t), replace "Append G to Returns(S_t, A_t); $Q(S_t, A_t) \leftarrow$ average(Returns(S_t, A_t))" with:

$$
\begin{aligned}
N(S_t, A_t) &\leftarrow N(S_t, A_t) + 1 \\
Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} \big[G - Q(S_t, A_t) \big] \\
\pi(S_t) &\leftarrow \arg\max_a Q(S_t, a)
\end{aligned}
$$

Question 2

(a)

Suppose every-visit MC was used instead of first-visit MC on the blackjack task. Would you expect the results to be very different?

No, the results would be identical. In blackjack, a state can never be visited twice in the same episode, so "first visit" and "every visit" are always the same visit. This is because every hit either raises your sum or uses up your usable ace, so a state can never repeat.

(b)

What are the first-visit and every-visit estimators of the value of the nonterminal state?

The episode visits the nonterminal state s at times $t = 0, 1, \ldots, 9$, and every transition gives a reward of $+1$. With $\gamma = 1$ (no discounting), the return from time t is the number of remaining rewards:

$$G_t = 10 - t, \qquad \text{so the returns are } 10, 9, \ldots, 1.$$

First-visit estimator: use only the return from the first visit ($t = 0$), so we have 10 remaining rewards:

$$V(s) = G_0 = 10$$

Every-visit estimator: average the returns from all 10 visits. The sum of the returns is

$$10 + 9 + \cdots + 1 = 55,$$

then we divide by the number of visits:

$$V(s) = \frac{55}{10} = 5.5$$

Question 3

(a)

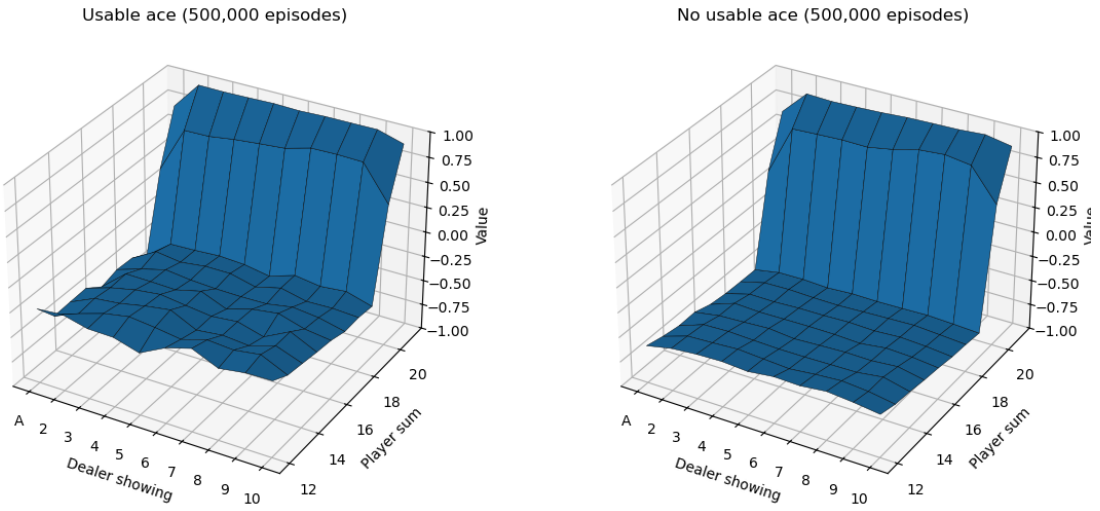


Figure 1: First-visit Monte Carlo policy evaluation (reproduction of Figure 5.1, 500,000 episodes).

(b)

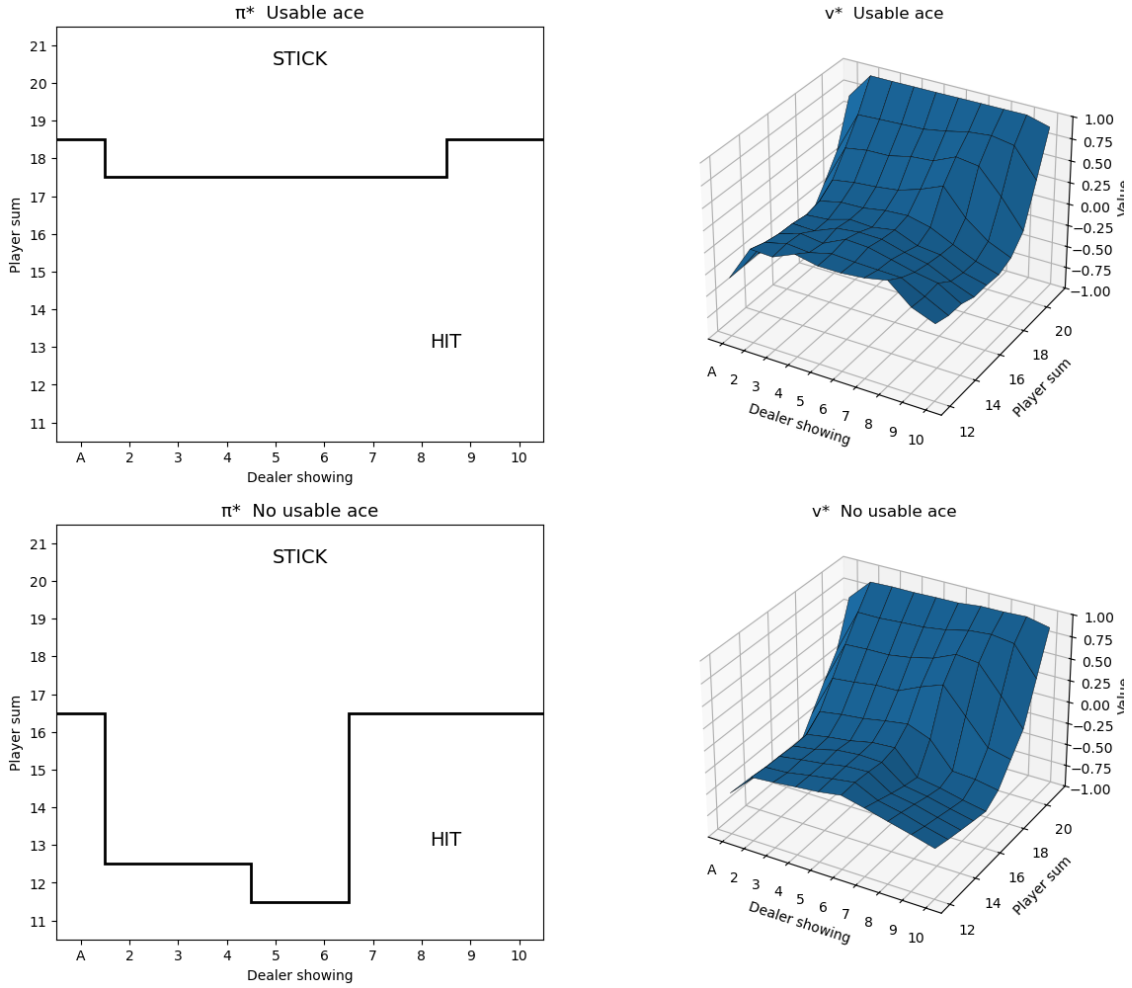


Figure 2: First-visit Monte Carlo control with exploring starts (reproduction of Figure 5.2).

Question 4

(a)

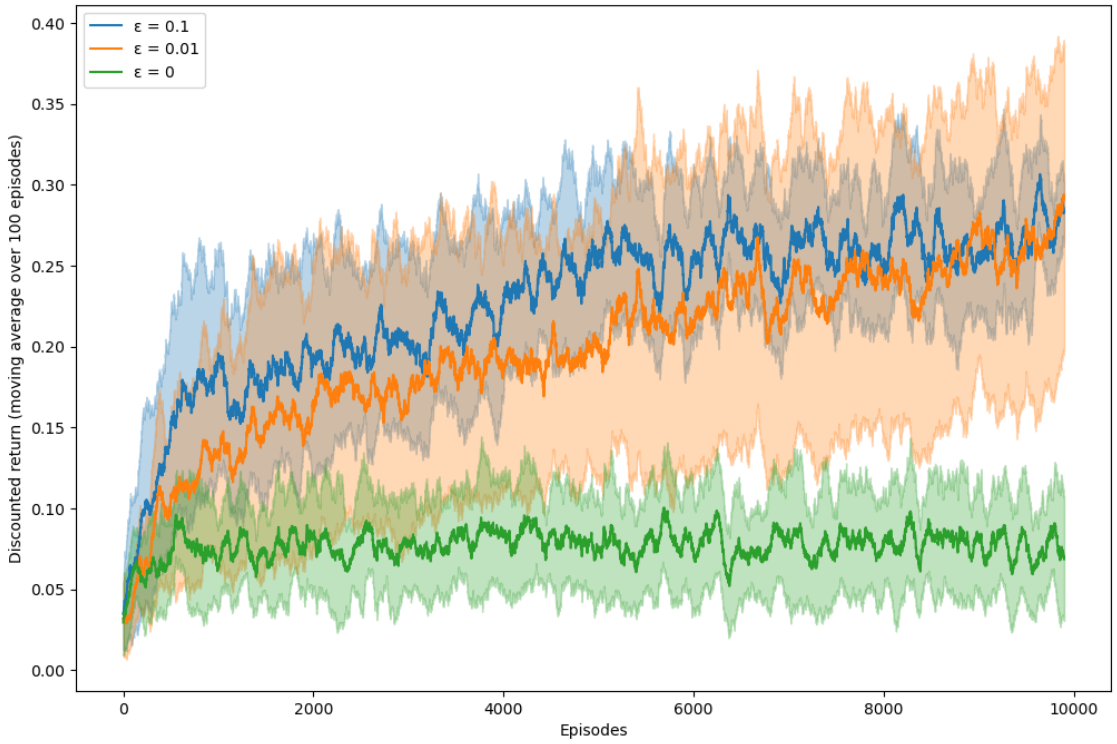


Figure 3: On-policy first-visit Monte Carlo control on FrozenLake for $\varepsilon = 0.1, 0.01, 0$ (10 trials, 10^4 episodes, moving average over 100 episodes).

(b)

Explain how the results of the $\varepsilon = 0$ setting demonstrate the importance of doing exploring starts in Monte Carlo ES.

With $\varepsilon = 0$, the curve remains flat around 0.08 for all 10,000 episodes. All Q values start at 0, so at first ties are broken randomly and the agent doesn't learn. As soon as one action leads to the goal, its Q value becomes positive and therefore becomes the greedy choice in that state. From then on, the agent only picks that action and never tries any of the others. Monte Carlo ES avoids this problem with exploring starts: every episode begins from a randomly chosen state-action pair, so every pair keeps being tried and its Q value keeps getting updated.

Question 5

(a)

On-policy first-visit Monte Carlo control with $\varepsilon = 0.1$ on both racetracks (10 trials, 2000 episodes per trial). The shaded region is the 95% confidence band over trials.

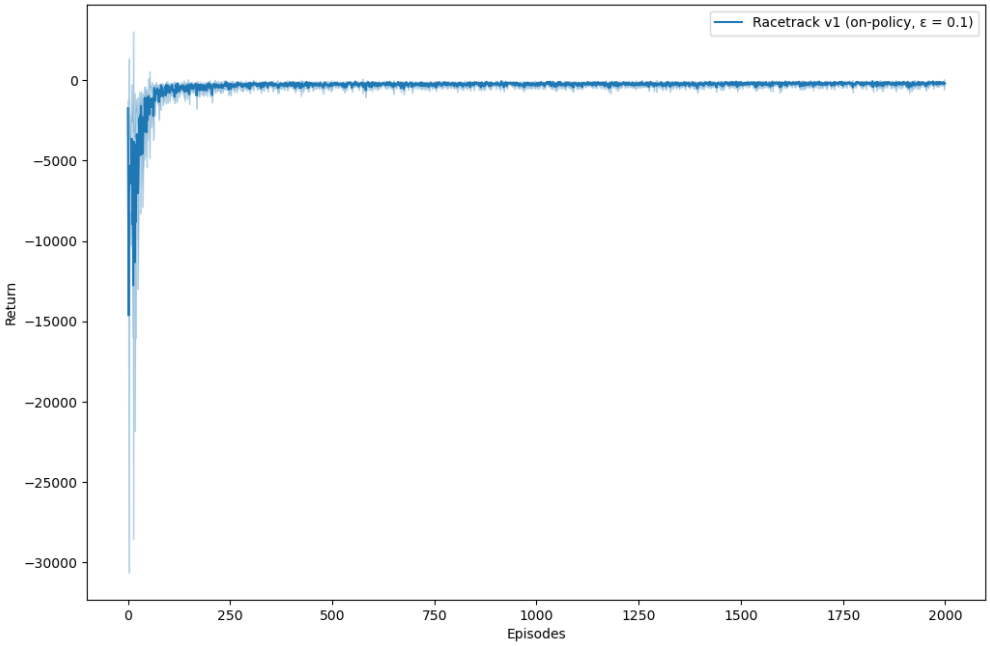


Figure 4: On-policy Monte Carlo control on Racetrack v1 (left track).

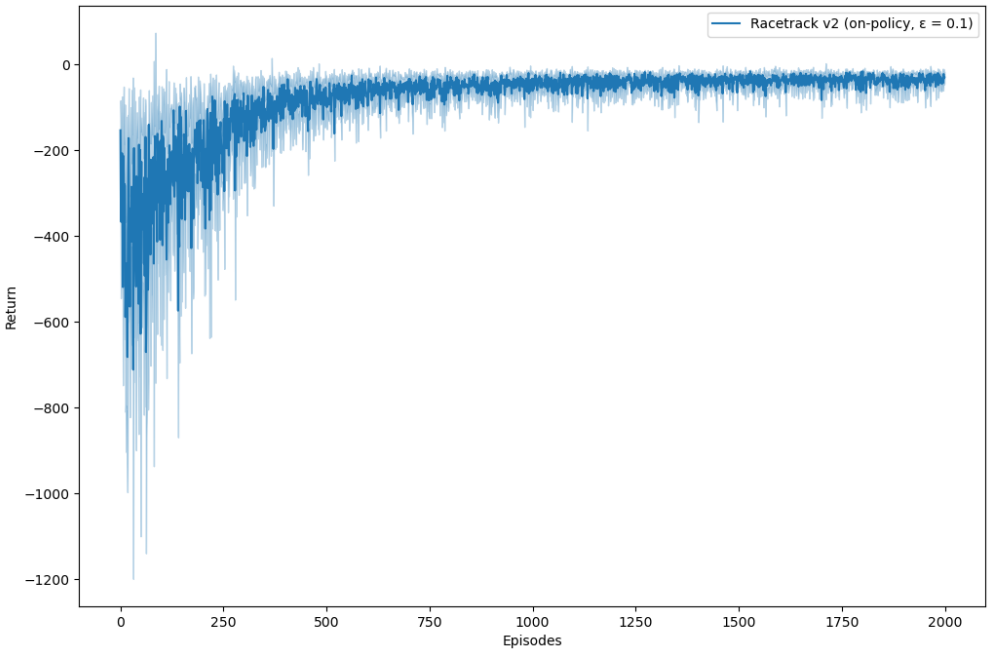


Figure 5: On-policy Monte Carlo control on Racetrack v2 (right track).

(b)

Off-policy Monte Carlo control with weighted importance sampling on Racetrack v2 (10 trials, 2000 episodes per trial).

Behavior policy: ε-greedy ($\varepsilon = 0.1$) with respect to the latest estimate of $Q(s, a)$, with ties broken at random.

Target policy: greedy with respect to $Q(s, a)$. After each training episode, one rollout of the greedy target policy is collected only for evaluation (capped at 1000 steps), and its return is plotted.

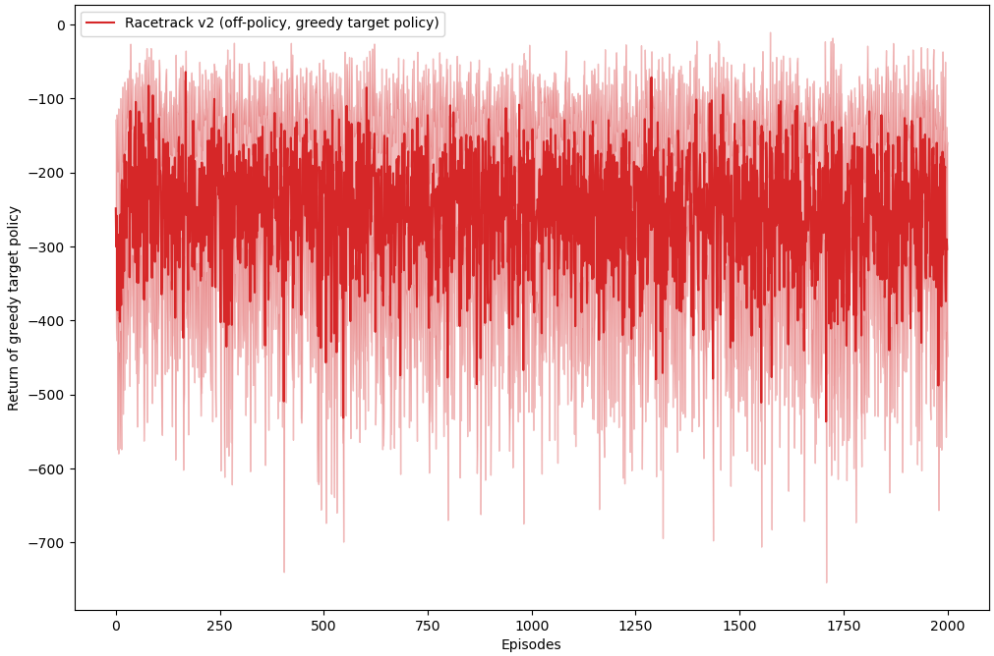


Figure 6: Return of the greedy target policy for off-policy Monte Carlo control on Racetrack v2.

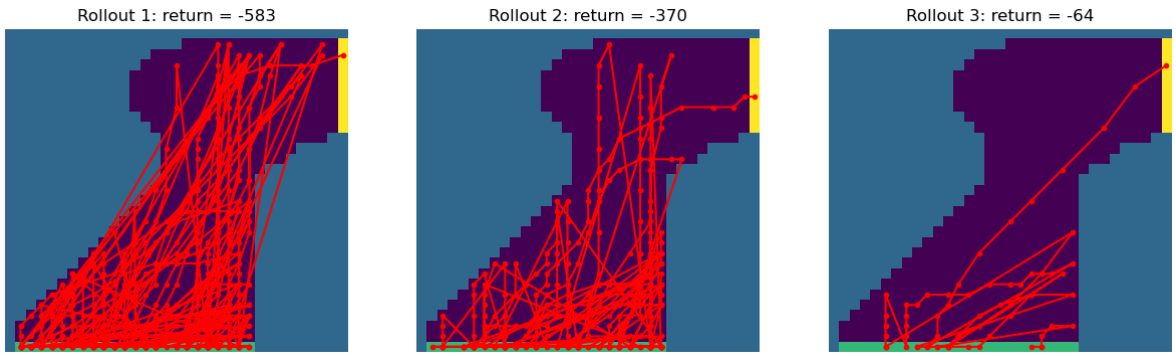


Figure 7: Several rollouts of the learned target policy on Racetrack v2.

(c)

Do you observe any significant differences between the on-policy and off-policy methods? Are there any interesting differences between the two racetracks?

Yes. On Racetrack v2, on-policy MC improved from about -400 to -36 per episode, while the greedy target policy from off-policy MC stayed around -250 the whole time. An off-policy MC with a greedy target only learns from the tail of every episode. It goes backwards and stops at the first action the greedy policy wouldn't have taken, due to the importance sample ratio being 0. With long episodes and $\varepsilon = 0.1$, only

the last few steps are learned. In addition, starting Q at 0 makes this worse, since every episode return is negative, so an updated action looks immediately worse than an untried action, and the loop stops very quickly.